

Part 2: Databricks Warehouses

Part 1: [Databricks Serverless Costs](#)

Databricks Warehouses are a group of SQL-optimized compute clusters. Available warehouses are listed under the *SQL Warehouses* sidebar of the Databricks workspace UI. They are the underlying compute type for

- Query Editor
- Catalog Explorer
- external BI/visualisation tools like PowerBI
- native Databricks dashboards

SQL Warehouses can also be selected as compute resource for

- Notebooks with SQL cells
- SQL tasks in Jobs
- Pipelines

Three Warehouse **types** exist, namely *Traditional*, *Pro*, and *Serverless*. They differ along a number of performance and behavioural dimensions:

	Classic	Pro	Serverless
Costs	DBU costs + cloud vendor costs		only DBU costs
Launch Time	slower bootstrapping phases, typically a few minutes		rapid, a few seconds
Scaling Behaviour	slower up/downscaling, heuristic-based autoscaler		faster up/downscaling, ML-based autoscaler
Performance Features	Photon	Photon + selective read/update operations	

By default, a small serverless warehouse gets created in a Databricks workspace.

SQL Warehouses support **horizontal** as well as **vertical scaling** strategies in the form of cluster groups and warehouse sizes:

- Warehouses have a static size of one cluster by default. To process more queries **concurrently**, the *Maximum Clusters* property in the *Scaling* setting can be increased. This allows the allocation of additional warehouse clusters of the same size during phases with high query load. As a general heuristic, a single warehouse cluster is suitable for up to ten concurrent user queries. The following SQL snippet aggregates the total time each warehouse spends on its **concurrency levels** during the previous three months:

```
WITH time_spent AS (  
  SELECT *, TIMESTAMPDIFF(SECOND, event_time, LEAD(event_time) OVER  
    (PARTITION BY account_id, workspace_id, warehouse_id ORDER BY event_time)) duration_sec  
  FROM system.compute.warehouse_events  
  WHERE event_time >= CURRENT_DATE() - INTERVAL 3 MONTHS  
)  
SELECT warehouse_id, cluster_count, ROUND(SUM(duration_sec) / 60, 2) minutes_running,  
CONCAT(account_id, '#', workspace_id) workspace  
FROM time_spent WHERE cluster_count > 0 GROUP BY ALL  
ORDER BY workspace, warehouse_id, minutes_running DESC
```

- Different **size categories** determine the processing power of a single warehouse cluster. Larger warehouse sizes do not change the worker types but allocate more nodes. The instance type of worker nodes is *i3.2xlarge* on AWS, *Standard_E8ds_v4* on Azure, and *n2-highmem-8* on GCP

T-Shirt Size	Workers Count	Total Worker Resources
2X-Small	1	8 vCPU, ~64 GB Memory
X-Small	2	16 vCPU, ~128 GB Memory
Small	4	32 vCPU, ~256 GB Memory
Medium	8	64 vCPU, ~512 GB Memory
Large	16	128 vCPU, ~1024 GB Memory
X-Large	32	256 vCPU, ~2048 GB Memory
2X-Large	64	512 vCPU, ~4096 GB Memory
3X-Large	128	1024 vCPU, ~8192 GB Memory
4X-Large	256	2048 vCPU, ~16384 GB Memory
5X-Large	512	4096 vCPU, ~32768 GB Memory

made by  / [bigdatadude](#)

The total warehouse **costs** and **DBU** consumption during the last three months can be determined with the following system table query:

```
WITH wh_costs AS (  
  SELECT CONCAT(u.account_id, '#', workspace_id) workspace, u.usage_metadata.warehouse_id, usage_quantity DBUs,  
    (pricing.effective_list.default * usage_quantity) costs, currency_code currency  
  FROM system.billing.usage u JOIN system.billing.list_prices lp  
  ON u.sku_name = lp.sku_name AND price_start_time <= usage_start_time AND (price_end_time IS NULL  
  OR price_end_time >= usage_start_time)  
  WHERE u.usage_metadata.warehouse_id IS NOT NULL AND usage_date >= CURRENT_DATE() - INTERVAL 3 MONTHS  
)  
SELECT warehouse_id, SUM(costs) total_cost, currency, SUM(DBUs) total_DBUs, workspace  
FROM wh_costs GROUP BY ALL
```

[This page](#) describes Databricks **system tables** in more detail.

The SQL snippet below prints the latest **configurations** of all active SQL warehouses:

```
WITH wh_configs AS (  
  SELECT *, ROW_NUMBER() OVER (PARTITION BY account_id, workspace_id, warehouse_id ORDER BY change_time DESC) change  
  FROM system.compute.warehouses QUALIFY change = 1  
)  
SELECT * FROM wh_configs WHERE delete_time IS NULL
```

Warehouses can be active but **idle**, incurring costs without executing any queries. If longer idle phases are observed, users should consider setting/reducing the **Auto stop** time in the warehouse configuration or reducing the t-shirt size.

A query proposal for determining the total "busy" and "idle" times per warehouse during a predefined interval is pasted below. The column **minutes_active_busy** is derived by aggregating intervals that overlap with query durations and during which a warehouse is active (cluster_count > 0), **minutes_active_but_idle** is defined as active periods that cannot be mapped to any queries from the `system.query.history` table:

```
DECLARE OR REPLACE VARIABLE interv_start STRING DEFAULT '2026-01-01'; -- adjust timeframe start  
DECLARE OR REPLACE VARIABLE interv_end STRING DEFAULT '2026-03-15'; -- adjust timeframe end  
  
WITH wh_queries AS ( -- warehouse queries within timeframe  
  SELECT account_id, workspace_id, compute.warehouse_id, statement_id, start_time, end_time  
  FROM system.query.history WHERE compute.warehouse_id IS NOT NULL AND execution_duration_ms IS NOT NULL  
  AND start_time < end_time AND start_time >= interv_start AND end_time <= interv_end  
, wh_events AS ( -- warehouse events with their subsequent event timestamps  
  SELECT account_id, workspace_id, warehouse_id, event_time, LEAD(event_time) OVER (PARTITION BY account_id, workspace_id, warehouse_id  
  ORDER BY event_time) next_event, cluster_count  
  FROM system.compute.warehouse_events  
  WHERE event_time BETWEEN(interv_start) AND (interv_end)  
, running_events AS ( -- warehouse events other than starting & stopped with overlapping queries  
  SELECT * EXCEPT (q.account_id, q.workspace_id, q.warehouse_id)  
  FROM wh_events e LEFT JOIN wh_queries q ON q.account_id = e.account_id AND q.workspace_id = e.workspace_id  
  AND q.warehouse_id = e.warehouse_id AND start_time < next_event AND end_time > event_time WHERE cluster_count > 0  
, queried_events AS ( -- one second slots with overlapping queries  
  SELECT CONCAT(account_id, '#', workspace_id) workspace, warehouse_id, explode(sequence(event_time, next_event, INTERVAL 1 SECOND)) slot_ts,  
    date_trunc('SECOND', slot_ts) slot_sec, CASE WHEN start_time <= slot_ts AND end_time >= slot_ts THEN 1 ELSE 0 END covered  
  FROM running_events  
, agg_slots AS ( -- aggregating executed queries of seconds slots  
  SELECT workspace, warehouse_id, slot_sec, SUM(covered) exec_queries FROM queried_events GROUP BY ALL  
)  
SELECT warehouse_id, ROUND((COUNT(exec_queries) FILTER (WHERE exec_queries = 0)) / 60, 2) minutes_active_but_idle,  
ROUND((COUNT(exec_queries) FILTER(WHERE exec_queries > 0)) / 60, 2) minutes_active_busy, workspace  
FROM agg_slots GROUP BY ALL
```

More optimization queries can be found [here](#).